DEPLOYMENT CONCEPT · NOT BUILT

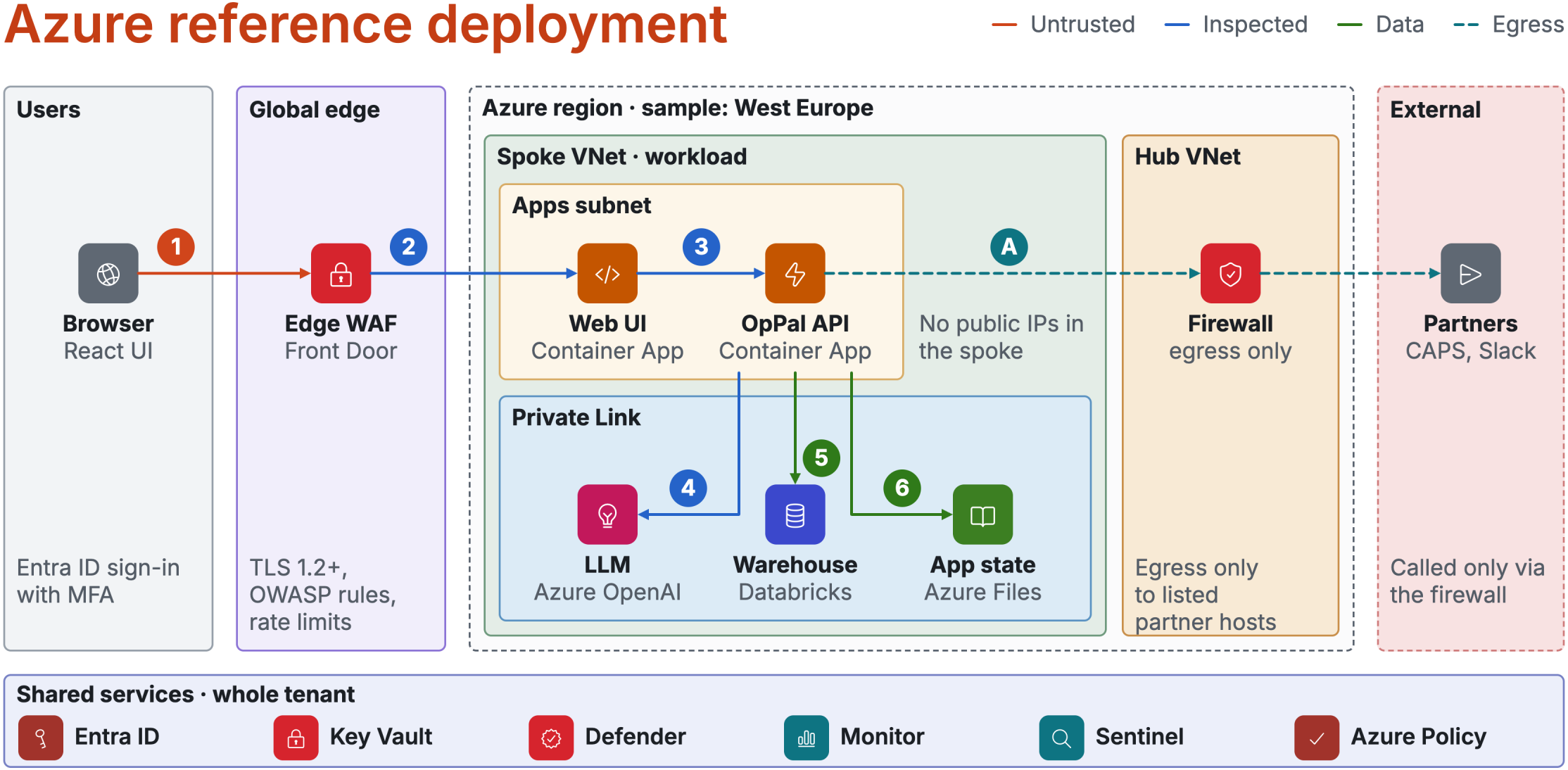
OpPal on Azure

An imagined reference deployment: one region, private by default, every OpPal control kept and Azure's layered around it.



Concept for OpPal v14.14.0 · October 2026 · region and names are samples

Azure reference deployment



One request on Azure, hop by hop

Step	Hop	Control at the crossing
1	Browser → Edge WAF	HTTPS with TLS 1.2+; OWASP managed rules and rate limits
2	Edge WAF → Web UI	Private Link origin; the apps environment has no public ingress
3	Web UI → OpPal API	Internal ingress only; the API still checks every bearer token
4	OpPal API → Azure OpenAI	Private endpoint; PHI scrubbed first; key held in Key Vault
5	OpPal API → Databricks	Private Link; scoped, read-only SQL through the shipped connector
6	OpPal API → Azure Files	Private endpoint; ledger, cases, settings and Brain persist here
A	OpPal API → Firewall → Partners	Only listed partner hosts pass the hub firewall; dry-run by default

Sign-in happens before step 1: Entra ID issues the token the API validates. Every hop's logs stream to Log Analytics.

Where each part of OpPal runs

OpPal today	Azure service	How it fits
Web UI (nginx, React build)	Container Apps	Internal ingress behind Front Door
OpPal API (FastAPI, port 8080)	Container Apps	One replica; probes /api/auth/users
Container images	Container Registry	Private; Defender scans every push
Warehouse (oppal.db today)	Azure Databricks SQL	Through the shipped Databricks connector
app/data and app/semantic	Azure Files	Mounted where compose mounts them today
LLM provider	Azure OpenAI	The shipped “azure” provider
Semantic cache	Azure Cache for Redis	Optional: set REDIS_URL, else in-process
Secrets (auth, webhook, SMTP)	Key Vault	Read by the apps’ managed identity
Email notifications	Communication Services	SMTP relay the email adapter speaks
Logs and alerts	Log Analytics, Sentinel	Container logs plus ledger alerts
Network	Hub-spoke VNets, Firewall	No public IPs; egress through the hub

Azure adds layers around the app’s own checks

Azure, outside the app

- **Front Door WAF.** OWASP rules and rate limits before traffic reaches the region.
- **Private Link.** The apps environment and every data service have no public endpoint.
- **Entra ID.** Single sign-on with MFA; officers elevate through time-boxed PIM.
- **Managed identity.** The apps read Key Vault, Files and the registry with no stored keys.
- **Azure Policy.** Denies public endpoints and unencrypted storage at deploy time.

The app’s own, unchanged

- **Middleware.** Any /api call without a valid token gets a 401.
- **_user().** Re-resolves the principal from the token, with no header fallback.
- **_require().** Checks the route’s permission and logs every refusal.
- **MCP execute().** Final; pushes the caller’s scope into every query.
- **Two officers.** Map to two Entra accounts; never give one person both.

Where OpPal’s state lives on Azure

Store	Today	On Azure
Warehouse	app/data/oppal.db	Databricks SQL warehouse
Audit ledger	audit_log.jsonl, ledger_head.json	Azure Files, plus a copy in immutable Blob
Cases, actions, history	SQLite files under app/data	Azure Files share
Settings	settings.json	Azure Files; secrets move to Key Vault
Context Brain	semantic/domains/*.md, versions	Azure Files, seeded from the image

Ceiling: one API replica. SQLite files and a ledger lock file on an SMB share can’t safely take two writers. Scaling out means moving these stores to Azure Database for PostgreSQL first.

Off-box ledger copy. Immutable Blob storage with time-based retention keeps a copy of the hash chain that nobody, admins included, can rewrite.

What the code would need

Change	Where	Rough size
Validate Entra ID tokens; map groups to roles and scopes	app/security/auth.py	Medium
Let Key Vault env vars win over settings.json	app/config.py	Small
Managed-identity tokens for Azure OpenAI, not an api-key	app/llm/base.py	Small
Port SQLite date functions in Brain SQL to Databricks SQL	domains/*/policy_rules.md	Medium
Copy each ledger entry to immutable Blob storage	app/trust/ledger_v2.py	Medium
Run run_all_tests.sh before any image reaches the registry	CI pipeline	Small

One rule holds: don't trust Container Apps' sign-in headers. _user() has no header fallback on purpose, so the API validates the Entra token itself.